

Malfunctions in Two Flatland Encodings

Raphael Kluge and Ashik Chandra Das
Group Name: Arrival Delayed by 1 Hour

Faculty of Computer Science, University of Potsdam

Abstract. Railway scheduling is very important for effective train operations by minimizing conflicts and delays. However, disruptions such as malfunctions or track blockages need rescheduling. This study explores Answer Set Programming (ASP) for railway rescheduling, integrating it with Flatland. We develop two encoding approaches: Cell-based encoding, which models individual track cells, and Graph-based encoding, which abstracts intersections for better performance. We have implemented using Clingo and our method adjusts train movements to minimize disruptions. While the graph-based approach improves efficiency, it introduces complexity.

Keywords: Answer Set Programming (ASP), Clingo, Vehicle Rescheduling Problem, Flatland

1 Introduction

Railway scheduling is an increasingly important problem since there is a growing demand on railway networks. A proper railway schedule ensures that trains run efficiently while avoiding conflicts and delays. Basically, it involves planning paths, assigning time slots and coordinating train movements through the rail network. As railway systems grow with increasing demand, automated scheduling solutions are needed to optimize train operations. It is also necessary to ensure smooth and efficient train operations when there may be disruptions such as delays, track blockage or equipment failures. Without a proper rescheduling system, this kind of disruptions can cause major delays, unhappy customers with a poor running system. Modern rail networks are complex with interconnected routes and high traffic density. As a result, even a minor disturbance can impact multiple trains where real time adjustments to schedule is required. Railway scheduling and rescheduling techniques help to minimize this disturbance by reducing conflicts, optimising train movements and improving the overall system. In this report, we have discussed related literature, different algorithms, our approaches to find optimal states, and implementations of our work.

2 Related Literature

Railway scheduling and rescheduling has been studied using various optimization and AI-based approaches. [2] have reviewed different strategies for handling train delays and disruptions, highlighting mathematical and algorithmic

solutions. More recently, AI techniques such as reinforcement learning [5] and constraint-based optimization have been used to improve train scheduling. [1] have explored graph-based optimization techniques for railway scheduling, focusing on track allocation to minimize conflicts and improve efficiency. Also, [6] utilized graph theory to address train scheduling challenges, identifying optimal routes, reduction of wait times, and optimization of resource utilization. Their approach highlights the importance of mathematical models in handling complex scheduling constraints within railway networks.

3 Malfunction/ Rescheduling

Definition: Vehicle Rescheduling Problem was defined in [3] as follows: “The vehicle rescheduling problem (VRSP) arises when a previously assigned trip is disrupted. A traffic accident, a medical emergency and a breakdown of a vehicle are examples of possible disruptions that demand the rescheduling of vehicle trips. The VRSP can be approached as a dynamic version of the classical vehicle scheduling problem (VSP) where assignments are generated dynamically.”

Known Algorithms: Different algorithms have been used to solve the rescheduling problem. [3] have discussed some auction based algorithms for the Single-Depot rescheduling problem. The Sequential Auction Algorithm optimizes vehicle scheduling by repeatedly adjusting trip assignments and costs across different possible routes. The Parallel Auction Algorithm which is based on the Jacobi method, speeds up rescheduling by using multiple processors to handle tasks simultaneously, reducing wait times between them. To increase optimization speed, the Common Feasible Network Approach algorithm initially solves a reduced network, excluding interrupted trips and backup options. Also, Mixed-Integer Linear Programming (MILP) and Constraint Programming (CP) are common methods for train rescheduling. MILP uses math equations to model train schedules and track use by finding optimal solutions for small to medium networks. [2] has applied MILP to fix track conflicts in real-time. On the other hand, CP uses rules and constraints to manage train movements and delays. It is good for handling strict scheduling rules like train spacing. [4] have used CP to optimize train rescheduling.

4 Our Approaches

Several approaches can be used to find an optimal state. This report, we explore railway rescheduling using Answer Set Programming (ASP) which is a logic-based approach for solving complex rescheduling problems. Due to its problem-solving capabilities, this method is suitable for real time train schedule adjustment. On the other hand, Flatland is used for the simulation of the railway scheduling problems. By combining ASP’s problem-solving capabilities with Flatland’s simulation environment, we have tested rescheduling strategies that minimize conflicts, optimize train movement, and improve overall railway operations. We have used Clingo for solving. The Python API makes it easy to use

Clingo to define constraints and optimization rules in a declarative programming style.

5 Railway Network

The railway network is a 2D grid, with cells indexed by (row, column) coordinates, starting from (0, 0) at the top-left. The trains are introduced as Agent. In the railway network grid, there are different track types which make a complete railway network. Other cells are filled with trees, hills or open spaces. Agents are directed on the cell to four different cardinal directions (North, East, South, West). Depending on their incoming direction, there are eight different track types which are used by the agents to reach their destination. Here, each cell is converted into a bitmap of 16 bits. As an example, the figure below indicates a track with its resulting bitmap.



	N	E	S	W
N	1	0	0	0
E	1	1	0	0
S	0	0	1	1
W	0	0	0	1

Fig. 1. Example of a track and its corresponding bitmap

For this track type, the bitmap is 1000110000110001

6 Clingo Encodings

We did two main encodings, a cell-based and a graph-based approach. While the first one was relatively easy to convert into actions, the latter was a lot more complicated. However, the performances were better in the graph-based approach.

6.1 Cell-Based Encoding

```

1 %init train occupations on second time step
2 occupation(train(ID), (X,Y), Dep+1, Dir) :- start(ID, (
    X,Y), Dep, Dir).
```

```

3 %and action of the past
4 action(train(ID), wait, 0..Dep) :- start(ID, _, Dep, _)
.
5 action(train(ID), move_forward, Dep+1) :- start(ID, _,
    Dep, _), not action(train(ID), wait, Dep+1).

```

Occupation/4 atoms are of the same format as start/4 and show the train's position during its journey. Action/3 atoms are also generated for the start and are used later with the occupation/4 to solve the configuration.

```

1 % If there is a track_from_to with DirFrom equals DirTo
  , then we can move forward
2 { action(train(ID), move_forward, T+1) } :-
3 track_from_to(TrackNum, DirFrom, DirTo),
4 cell((X,Y), TrackNum),
5 occupation(train(ID), (X,Y), T, DirFrom),
6 DirFrom = DirTo, end(ID, _, Arr), T < Arr.

```

Actions are dynamically generated for each occupation. The move action depends on the direction the train was at the start of the action vs. the direction afterwards. For each track type, there exists one or two possible “DirTo” directions for each “DirFrom” direction, as described in section 6). Infinite models are prevented by the statement $T < Arr$, so no trains can move after their expected arrival.

6.2 Graph-Based Encoding

```

1 intersection((X,Y), -1) :- cell((X,Y), Track), Track
    !=0, Track!=32800, Track!=1025, Track!=4608, Track
    !=16386, Track!=72, Track!=2064.
2 intersection((X,Y), ID) :- start(ID,(X,Y), _, _).
3 intersection((X,Y), ID) :- end(ID,(X,Y), _).

```

In our graph-based encoding we abstracted the environment by only considering intersection type cells, basically all track types except type #1. Also we made the start and end cell of a train an intersection.

```

1 xy_change_direction(1,0,n,3).
2 xy_change_direction(0,-1,e,2).
3 xy_change_direction(-1,0,s,1).
4 xy_change_direction(0,1,w,0).
5
6 section_cell((X1,Y1), (X2,Y2), Track2, DirFrom, TID) :-
7 cell((X1,Y1), Track1), intersection((X1,Y1), TID),
8 cell((X2,Y2), Track2), not intersection((X2,Y2), _),
9 track_from_to(Track1, DirFrom, DirTo),
10 xy_change_direction(X1-X2,Y1-Y2,DirTo,_).

```

In order to connect cells to the intersections, we created an `xy_change_direction/3` atom, which takes both the X and Y differences of two cells and selects the direction of the move. This creates so-called “section_cells”. These cells are then incrementally added, like pearls to a chain, until they reach the next intersection.

```

1 section_cell(A, B, C, Track, DirFrom1, DirTo, TID1) :-
2   section_cell(A, C, Track, DirFrom1, TID1),
3   section_cell(B, C, Track, DirFrom2, TID2),
4   arriving_direction(A, B, DirTo), A!=B.
5
6 intersection(A, B, DirFrom, DirTo, TID, SCC+1) :-
7   SCC = #count{ C: section_cell(A, B, C, _, DirFrom,
8     DirTo, TID) },
9   section_cell(A, B, _, _, DirFrom, DirTo, TID).

```

From both sides, the `section_cell/5` atoms are then combined into the `section_cell/7` atom in order to close the chain. Counting all cells in a connection between two intersections results in an `intersection/6` atom, which is used later to let trains move between the intersection cells.

```

1 move(TID, (X1,Y1), B, Dep, Dep + Duration, DirFrom,
2   DirTo) :-
3   start(TID, (X1,Y1), Dep-2, DirFrom),
4   section_cell((X1,Y1), B, (X2,Y2), _, _, _, _),
5   xy_change_direction(X1-X2, Y1-Y2, _, _),
6   intersection((X1,Y1), B, DirFrom, DirTo, TID,
7     Duration).

```

Using the `start/4` and the `intersection/6` atoms, a first `move/7` atom is created and moves the train to the next (and only possible) intersection. Note that we had to use `Dep-2` in order to satisfy the way Flatland works in regard to when actions are scheduled, as is the case there, versus directly executed as we do it. The `xy_change_direction/4` atom is necessary to check that the `section_cell` borders the start cell.

```

1 { move(TID, (X1,Y1), B, Arr, Arr + Duration, DirFrom,
2   DirTo) } :-
3   move(TID, X, (X1,Y1), Dep, Arr, _, _),
4   intersection(X, (X1,Y1), _, DirFrom, _, _),
5   section_cell((X1,Y1), B, (X2,Y2), _, _, _, _),
6   xy_change_direction(X1-X2, Y1-Y2, _, _),
7   intersection((X1,Y1), B, DirFrom, DirTo, _,
8     Duration),
9   end(TID, _, ReqArr), Arr + Duration <= ReqArr.

```

Now, using our intersection map we were able to create more consecutive moves. Again, the model count is limited by the expected arrival time in the

end/3 atom. Through integrity constraints we enforced an arrival at the end cell, only one move at a time as well as avoiding train collisions.

```

1 % left is move_left
2 action(train(TID), move_left, Dep) :-
3     move(TID, (X1,Y1), B, Dep, Arr, DirFrom, DirTo),
4     intersection((X1,Y1), B, _, _, _, Dur),
5     section_cell((X1,Y1), B, (X2,Y2), _, DirFrom, _, _)
6     ,
7     cell((X1,Y1), TrackNum),
8     xy_change_direction(X1-X2, Y1-Y2, DirTo, ToVal),
9     direction(DirFrom, FromVal),
10    (FromVal - ToVal + 4) \ 4 = 3,
    DTC = #count{ DirToC: track_from_to(TrackNum,
        DirFrom, DirToC) }, DTC > 1.

```

Actions are generated in a similar manner to the cell-based encoding. We now want to illustrate an example of this conversion from moves to actions when entering a section. This depends both on the change of direction (left, right, forward) as well as the number of possible directions, because in Flatland, if there is only one direction possible, it is a `move_forward` action regardless of the actual direction. The example above is an actual `move_left`, because there are more than one possible direction for the combination of track type and incoming direction.

We use the “value” of the cardinal directions ($n=3, e=2, s=1, w=0$) to calculate the possible move directions in line 9. In this case, we compare to 3, which equals a `move_left` (e.g., $FromVal=3, ToVal=0$, a shift from facing north to facing west is a left turn). This is done for all possible directions, and if $DTC=1$, it is always a `move_forward`.

```

1 action(train(TID), move_forward, Dep+1..Arr-1) :-
2     move(TID, X, A, _, Dep, _, _), move(TID, A, B, _,
3     Arr, _, _).
4 action(train(TID), move_forward, Dep+1..Arr-1) :-
5     start(TID, A, Dep-1, _), move(TID, A, B, _, Arr, _,
6     _).

```

The good thing about the graph-based encoding is that we just have to fill each action in a section with a `move_forward`.

```

1 #minimize { 1, TID, A, T: action(TID, A, T) }.

```

In both encodings we are minimizing the total action count of all agents.

7 Implementation Challenges and Solutions

When the malfunction occurs, we have tried different approaches to get the optimal solution. Our initial approach was for all agents to find their shortest

path from their new current position after the malfunction occurred. This might however not be necessary for all trains, only the ones who were subject to the malfunction or any trains which cross their path. Building up on this we had the system remind non-conflicting trains while only re-configuring the other agents.

8 Malfunctions in our encodings

We used the Clingo Flatland repository as the basis and modified parts in the Python files as well as we did in our encodings.

8.1 Python API

```
1 facts.append(f'forced_action(train({key}},{actions[
    index][key]},{index}).\n')
```

In the method `convert_formers_to_clingo(actions)`, we appended the action atoms with a different name and as a fact instead of integrity constraints, as this works well with our encodings.

We did the same in the method `convert_malfunctions_to_clingo(all_malfs, new_malfs, timestep)`.

```
1 facts.append(f'forced_action(train({train}),wait,{t}).\n
    n')
```

Furthermore, we added an extra atom called `malfunction_extra_time` to allow the trains to exceed their arrival limit individually, without using the `global/1` atom.

```
1 for m in all_malfs:
2     train, duration = m[0], m[1]
3     malfunction_extra_times[train] =
4         malfunction_extra_times.get(train, 0) +
5         duration
6     print(f"malfunction_extra_times[{train}] += {
7         duration}")
8 for (train, duration) in malfunction_extra_times.items():
9     print(f'malfunction_extra_time({train}, {
10         duration})')
11 facts.append(f'malfunction_extra_time({train},
12     {duration}).\n')
```

8.2 In Cell-based

Here we first created the same `malfunction_extra_time` atom.

```
1 malfunction_extra_time(ID, 0) :- train(ID).
```

This extra time is then added to the time constraint in each action creation.

```
1 { action(train(ID), move_forward, T+1) } :-
2   track_from_to(TrackNum, DirFrom, DirTo),
3   cell((X,Y), TrackNum),
4   occupation(train(ID), (X,Y), T, DirFrom),
5   DirFrom = DirTo,
6   end(ID, _, Arr), malfunction_extra_time(ID, MET),
7   T < Arr + MET.
```

Here we simply convert `forced_actions` to `actions` in order to enforce them on the encoding. This is possible because all of them (except for the `start` action) have a relaxed cardinality constraint of 0 to 1.

```
1 action(TID, A, T) :- forced_action(TID, A, T).
```

The start action also now depends on whether there already is an enforced wait action or not.

```
1 action(train(ID), wait, 0..Dep) :-
2   start(ID, _, Dep, _).
3
4
5 action(train(ID), move_forward, Dep+1) :-
6   start(ID, _, Dep, _),
7   not action(train(ID), wait, Dep+1).
```

8.3 In Graph-Based

This is much more tricky to achieve than simple cell-based, because the conversion is much more complicated.

First we needed to check if a start had to be postponed. This had to be enforced either at the last known wait action or just before the first move action.

```
1 % check if train has to start by checking first forced
   move forward
2 start(ID, A, Start-1, Dir) :-
3   Start = #min { T: forced_action(train(ID),
4     move_forward, T) },
5   forced_action(train(ID), move_forward, _), start(ID,
6     A, _, Dir).
```

```

6  % if train has no forced move actions, start must be
   after last wait actions if it exists
7  start(ID, A, Start, Dir) :-
8      Start = #max { T: forced_action(train(ID), wait, T)
   },
9      not forced_action(train(ID), move_forward, _),
      start(ID, A, _, Dir).

```

Also we noted for each timestep and train, whether there is a malfunction during that time.

```

1  %is train disabled at T?
2  %is_malfunction(TID, T, MalDur)
3  { is_malfunction(TID, 0..EndT, 0) } :- global(EndT),
   train(TID).
4  is_malfunction(TID, T..T+Dur-1, Dur) :-
   clean_malfunction(TID, Dur, T).
5  % one is_malfunction per timestep
6  :- global(EndT), train(TID), not is_malfunction(TID,
   0..EndT, _).
7  :- train(TID), is_malfunction(TID, T, IsMal1),
   is_malfunction(TID, T, IsMal2), IsMal1!=IsMal2.

```

A problem arose when we realized that some malfunctions flatland gave were overlapping, which created problems, so we had to merge those that overlapped.

```

1  % we need to merge malfunctions that overlap or are
   included in other
2  malfunction_merge(TID, LastArr-T1, T1) :-
3      malfunction(TID, Dur1, T1), malfunction(TID, Dur2,
   T2),
4      LastArr = #max { Dur1+T1; Dur2+T2 },
5      (Dur1,T1)!=(Dur2,T2),
6      T1<=T2,
7      T1+Dur1>=T2.
8
9  overlap(TID, Dur1, T1, Dur2, T2) :-
10     malfunction(TID, Dur1, T1), malfunction(TID, Dur2,
   T2),
11     (Dur1,T1)!=(Dur2,T2),
12     T1<=T2,
13     T1+Dur1>=T2.
14
15 clean_malfunction(TID, Dur, T):- malfunction_merge(TID,
   Dur, T).
16 clean_malfunction(TID, Dur, T):-
17     malfunction(TID, Dur, T),

```

```

18     not overlap(TID, _, _, Dur, T),
19     not overlap(TID, Dur, T, _, _).

```

For the start we choose the latest start, because that one is the one enforced by malfunctions.

```

1  %first move from start
2  move(TID, (X1,Y1), B, LatestDep + 2, LatestDep + 2,
    Duration, StartDir, DirTo, Move) :-
3      start(TID, (X1,Y1), _, StartDir),
4      LatestDep = #max { Dep: start(TID, (X1,Y1), Dep,
    StartDir) },
5      intersection((X1,Y1), B, StartDir, DirTo, Move,
    Duration).

```

For the move generation we had to pay attention to an increase in duration time for each move due to malfunctions. Furthermore we added the individual grace period for each train equaling their total respective malfunction time count. We also had to be careful not just to simply add the malfunction duration of all malfunctions that happened during that time, but also include malfunctions that were reached in time by extending the duration of the move.

```

1  { move(TID, (X1,Y1), B, Arr, Arr, Duration, DirFrom,
    DirTo, Move) } :-
2      move(TID, X, (X1,Y1), Dep, Arr, 0, _, DirFrom, _),
3      intersection((X1,Y1), B, DirFrom, DirTo, Move,
    Duration),
4      end(TID, _, ReqArr),
5      Arr + Duration <= ReqArr + MalExTime,
6      malfunction_extra_time(TID, MalExTime).

```

We did this by recursively checking each timestep for malfunctions and counted down the duration if there was none, or otherwise went on to the next timestep.

```

1  % no mal at Arr
2  move(TID, A, B, Dep, Arr+1, MovesLeft-1, DirFrom, DirTo,
    Move) :-
3      is_malfunction(TID, Arr, Dur),
4      move(TID, A, B, Dep, Arr, MovesLeft, DirFrom, DirTo,
    Move),
5      Dur=0,
6      MovesLeft > 0.
7
8  % mal at Arr
9  move(TID, A, B, Dep, Arr+1, MovesLeft, DirFrom, DirTo,
    Move) :-
10     is_malfunction(TID, Arr, Dur),

```

```

11   move(TID, A, B, Dep, Arr, MovesLeft, DirFrom, DirTo
12       , Move),
13   Dur>0,
      MovesLeft > 0.

```

9 Result

Here, we can see the comparative graphical overview of the two approaches.

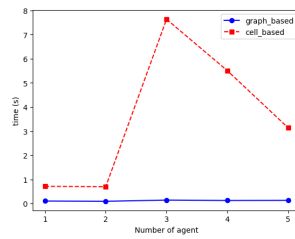


Fig. 2. Optimization time for different agents with 0 malfunction rate

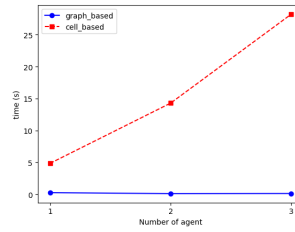


Fig. 3. Optimization time for different agents with 1/30 malfunction rate

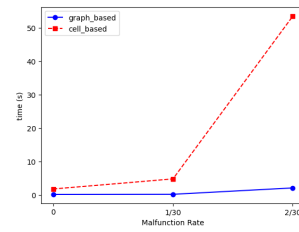


Fig. 4. Optimization time for different malfunction rates with 2 agents

In all cases, graph based approach needs the lower optimization time. Without any malfunction, both of the approaches take lower time but comparatively graph based approach needed less time. shows the time for the different malfunction rates.

10 Discussion

In this report we showed how we implemented our clingo encoding to solve flatland environments, specifically dealing with malfunctions. We had a lot of problems on the way and even afterwards there are still issues unresolved. Before we dealt with malfunctions, we had problems trying to implement the wait actions properly, because those create a lot more models which take way too much time to compute, especially in the cell-based encoding. We tried solving this for now by just optionally waiting one timestep at the start or not, which is of course not perfect. The graph-based implementation also proved very complicated, with more and more aspects added to the encoding, making it very hard to keep a working solution that satisfies every aspect, it also slowed down our progress. Some inputs from flatland were unexpected like overlapping malfunctions and although our actions/moves worked very well, the flatland output GIF did not always have the moves synchronized when dealing with malfunctions. We did not manage to implement more advanced ways of dealing with malfunctions,

like only rescheduling trains that were directly or indirectly affected. Also if we had time, we wanted to include other aspects, like time.

References

1. Borndörfer, R., Karbstein, M.: A direct connection approach to integrated lineplanning and passenger routing. Takustraße 7D-14195 Berlin-DahlemGermanyKonrad-Zuse-Zentrumfür Informationstechnik Berlin (2010)
2. Cacchiani, V., Huisman, D., Kidd, M., Kroon, L., Toth, P., Veelenturf, L., Wagenaar, J.: An overview of recovery models and algorithms for real-time railway rescheduling. *Transportation Research Part B: Methodological* (2014)
3. Jing-Quan Li, P.B.M., Borenstein, D.: The vehicle rescheduling problem: Model and algorithms. Wiley Inter Science (2007)
4. Mladenovic, S., Cangalovic, M.: Heuristic approach to train rescheduling. *Yugoslav Journal of Operations Research* (2007)
5. Mohanty, S., Nygren, E., Laurent, F., Schneider, M., Scheller, C., Bhattacharya, N., Watson, J., Egli, A., Eichenberger, C., Baumberger, C., Vienken, G., Sturm, I., Sartoretti, G., Spigler, G.: Flatland-rl: Multi-agent reinforcement learning on trains (2020)
6. Sitanggang, E., Prihandini, R.M., Agatha, A.B.: Graph theory: Its application in unraveling problems in modeling train scheduling problems (2024)